

Kolokwium 2 (1-ε) Java + ε C++

Imię:

Nazwisko:

Na napisanie testu jest 45 minut. Każde poprawnie rozwiązane zadanie to jeden punkt. Każde niepoprawnie rozwiązane zadanie to zero punktów (dotyczy to również zadań z wielokrotnym wyborem).

1. Jaki jest wynik uruchomienia następującego fragmentu kodu?

```
List<Integer> a = new ArrayList<Integer>();
List<String> b = new LinkedList<String>();
Set<Integer> c = new HashSet<Integer>();
Set<String> d = new TreeSet<String>();
System.out.println(a.equals(b)+" "+c.equals(d)+" "+
    a.equals(c)+" "+b.equals(d));
```

- a. true,true,true,true
 - b. true,true,false,false
 - c. false,false,true,true
 - d. false,false,false,false
2. Czy poniższy kod się kompiluje (jeśli nie wskaż miejsce i wytłumacz dlaczego)? Jaki jest wynik jego wykonania?

```
class A<T extends Throwable> {
    A() throws T { System.out.print("A"); }
}
class B extends A<Exception> {
    B() throws RuntimeException { System.out.print("B"); }
}
public class Main {
    public static void main(String[] args) throws Throwable {
        new B();
    }
}
```

3. Wybierz prawdziwe stwierdzenia:

- a. Jeśli funkcja `hashCode()` wywołana na dwóch obiektach ma tę samą wartość, to funkcja `equals()` musi zwracać że są równe.
- b. Jeśli funkcja `equals()` zwraca że dwa obiekty są równe, to muszą mieć tę samą wartość wywołanej na nich funkcji `hashCode()`.
- c. Funkcje `hashCode()` i `equals()` są metodami klasy `Object`.
- d. Aby poprawnie przechowywać obiekty w klasie implementującej interfejs `Collection` muszą one mieć poprawnie zdefiniowane funkcje `hashCode()` i `equals()`.

4. Klasa `A<T>` jest zdefiniowana następująco:

```
class A<T> { T id(T t) { return t; } }
```

zaznacz linie które skompilują i wykonają się poprawnie:

- a. `A<Integer> a = new A<Integer>(); Integer i = a.id(7);`
 - b. `A<? extends Integer> b = new A<Integer>();`
`Integer j = b.id(7);`
 - c. `A<? super Integer> c = new A<Integer>();`
`Object k = c.id(7);`
 - d. `A<?> d = new A<Integer>(); Object l = d.id(null);`
5. Wskaż operacje wykonywane przez wątek `TestThread`, które zostaną natychmiast przerwane w momencie gdy inny wątek wywoła `TestThread.interrupt()`:
- a. `TimeUnit.MILLISECONDS.sleep(300);`
 - b. `System.in.read();`
 - c. `wait();`
 - d. oczekiwanie na wejście do bloku `synchronized`.
6. Czy poniższy kod się kompiluje (jeśli nie wskaż miejsce i wytłumacz dlaczego)? Jaki jest wynik jego wykonania?

```
public class Zadanie {  
    public static void main(String[] args){  
        Collection<Integer> c = new LinkedList<Integer>();  
        for(int i = 99; i <= 102; i++){  
            c.add(i);  
        }  
        for(Integer i : c){  
            System.out.println("OK " + i);  
            if(i.equals(100)) c.remove(i);  
        }  
    }  
}
```

7. Czy poniższy kod się kompiluje (jeśli nie wskaż miejsce i wytłumacz dlaczego)? Jaki jest wynik jego wykonania?

```
public class Test {  
    public static void main(String[] args) {  
        Stream.generate(() -> 1).limit(3).  
            map(i -> i++).forEach(System.out::println);  
        Stream.generate(() -> Integer.valueOf(1)).limit(2).  
            map(i -> ++i).forEach(System.out::println);  
    }  
}
```

8. Jaki jest wynik działania następującego programu (zakładamy, że wszystkie importy są w porządku)?

```
class MyRunnable implements Runnable {
    public void run() {
        try { while(true) System.out.println("X"); }
        finally { System.out.println("Koniec"); }
    }
}

public class Main {
    public static void main(String[] args) throws
    InterruptedException {
        Thread t = new Thread(new MyRunnable());
        t.setDaemon(true);
        t.start();
        TimeUnit.SECONDS.sleep(1);
    }
}
```

- a. Będzie się wypisywać X w nieskończoność.
 - b. Przez sekundę będzie wypisywać się X, a potem wypisze się *Koniec* i program się zakończy.
 - c. Przez sekundę będzie wypisywać się X, a potem program zakończy się bez błędu.
 - d. Przez sekundę będzie wypisywać się X, a potem zostanie rzucony wyjątek i program się zakończy.
9. Które z poniższych zdań najtrafniej określa słowo kluczowe `synchronized`.
- a. Pozwala dwóm procesom na równoległe działanie w celu komunikowania się ze sobą.
 - b. Zapewnia dostęp do metody lub obiektu tylko jednemu wątkowi w danym momencie.
 - c. Zapewnia, że dwa lub więcej procesów wystartuje i zakończy się w tym samym momencie.
 - d. Zapewnia, że dwa lub więcej wątków wystartuje i zakończy się w tym samym momencie.

10. Czy poniższy kod może się zakleszczyć? Odpowiedź **dokładnie** uzasadnij.

```
public class T2 {
    public static void main(String[] args) {
        Lock[] l = new Lock[5];
        for (int i = 0; i < 5; i++) l[i] = new ReentrantLock();
        ExecutorService e = Executors.newFixedThreadPool(5);
        for (int i = 0; i < 5; i++)
            e.execute(new Philosopher(i,
```

```

        l[Integer.max(i, (i + 1) % 5)],
        l[Integer.min(i, (i + 1) % 5)]));
    e.shutdown();
}

public static class Philosopher implements Runnable {
    final Lock forkOne, forkTwo;
    final int id;
    public Philosopher(int id, Lock forkOne, Lock forkTwo) {
        this.forkOne = forkOne; this.forkTwo = forkTwo;
        this.id = id;
    }

    @Override
    public void run() {
        while (true) {
            forkOne.lock();
            try {
                System.out.println("Got one (" + id + ")");
                forkTwo.lock();
                try {
                    System.out.println("Got second and eating");
                } finally {
                    forkTwo.unlock();
                }
            } finally {
                forkOne.unlock();
            }
        }
    }
}
}

```

11. Na czym polega MVC?

.....

.....

.....

.....

.....

.....

12. Czy poniższy kod się kompiluje (jeśli nie wskaż miejsce i wytłumacz dlaczego)? Jaki jest wynik jego wykonania?

```

public class Sets {
    public static void main(String[] args) {
        Set<Byte> set = new HashSet<>();
        byte b = 0;
        set.add(b++);
        set.remove(b-1);
        set.add(b++);
        set.remove(b-1);
        set.add(b++);
        set.remove(b-1);
        System.out.println(set.size());
    }
}

```

13. Czy poniższy kod się kompiluje (jeśli nie wskaż miejsce i wytłumacz dlaczego)? Jaki jest wynik jego wykonania?

```

#include <iostream>
using namespace std;

struct int_list {
    int_list next;
    int i;
    int_list() { i = 0; }
};

int main() {
    int_list first;
    first.next = new int_list();
    first.next.next = new int_list{};
    first.next.next.next = nullptr;
    for(int_list * p = &first; p!=nullptr; p = p->next)
        cout << p->i << endl;
    return 0;
}

```

14. Jak jest różnica pomiędzy plikami nagłówkowymi (.h) i plikami (.cpp)

.....

.....

.....

.....

.....

.....

.....

15. Przeanalizuj poniższe deklaracje i wskaż, które instrukcje się **nie skompilują**.

```
int x;  
const int * a = &x;  
int const * b = &x;  
int * const c = &x;
```

- a. `*a = 5;`
- b. `*c = *a;`
- c. `++c;`
- d. `*b = 3;`
- e. `b++;`